Assignment 3 - GROUP 11

Tasneer Omar - 419762
Jodi Ang - 419762

Assignment 3 - GROUP 11

Johnson & Johnson Quarterly Earnings Dataset

Group Members

- Tasneer Omar
- Jodi Ang 419762
- Yuan Wang 419762
- Yuanqing Zhang 419762

Loading required Libraries

```
library(fpp2)
```

```
## -- Attaching packages --------------------------------------- fpp2 2.4 --
## v ggplot2 3.3.3     v fma     2.4
## v forecast 8.14
## Warning: package 'ggplot2' was built under R version 4.0.3
## Warning: package 'forecast' was built under R version 4.0.3
## Warning: package 'fma' was built under R version 4.0.3
## Warning: package 'expsmooth' was built under R version 4.0.3
## Warning: package 'lubridate' was built under R version 4.0.3
## Warning: package 'tseries' was built under R version 4.0.3
## Warning: package 'aTSA' was built under R version 4.0.3
```

```
library(forecast)
```

```
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo            zoo
## Registered S3 methods overwritten by 'forecast':
##   method             from
##   autoplot.acf       ggfa
##   autoplot.bats      ggfa
##   autoplot.decomposed ggfa
##   autoplot.forecast  ggfa
##   autoplot.stl       ggfa
##   autoplot.ts        ggfa
##   fitted.fARIMA      ggfa
##   plot.bats          ggfa
##   plot.decomposed    ggfa
##   plot.stl           ggfa
##   print.ts           ggfa
##   summary.fARIMA     ggfa
## Warning: replacement dimensions in multivariate array not equal to length of replacement
```

```
library(tseries)
```

```
## Registered S3 methods overwritten by 'zoo':
##   method               from
##   as.Date.numeric      tsbox
##   as.Ops.yearmon       zoo
##   as.Ops.yearqua       zoo
##   as.character.yearmon zoo
##   as.character.yearqua zoo
##   as.data.frame.zoo    zoo
##   as.numeric.yearmon   zoo
##   as.numeric.yearqua   zoo
##   as.yearmon.numeric   zoo
##   as.yearqua.numeric   zoo
##   cycle.zoo            zoo
##   fortify.zoo          ggfa
##   format.yearmon       zoo
##   format.yearqua       zoo
##   grid.pretty.yearmon  zoo
##   grid.pretty.yearqua  zoo
##   lag.zoo              zoo
##   lines.zoo            zoo
##   mean.yearmon         zoo
##   mean.yearqua         zoo
##   plot.zoo             zoo
##   points.zoo           zoo
##   rbind.zoo            zoo
##   scale.zoo            zoo
##   scale.zoo<-          zoo
##   scale.zoo<-          zoo
##   time.zoo             zoo
##   tsbox                ggfa
##   tsbox                tsbox
##   tsbox                tsbox
##   tsbox                tsbox
```

```
[quarterly_ttm_eps_clean_df_1_year_plot_data_series_ttm_eps_1_to_1_year_0 2 2011]
```

2(d) INTERPRETATION OF ALPHA (α)

```
--- 2(d) INTERPRETATION OF ALPHA (α) ---
```

Interpretation: The trailing-twelve-month eps has been growing at a rate of 10.2 % weight on current value +57.7 % weight on past values - The negative sign indicates mean reversion, which is a stabilizing force.

2(e) Fit Holt's Linear Trend Model

```
--- 2(e) FIT HOLT'S LINEAR TREND MODEL ---

--- 2(e) FIT HOLT'S LINEAR TREND MODEL ---

Traceback (most recent call last):
  File "/usr/local/bin/python3.11", line 1, in <module>
AttributeError: 'Series' object has no attribute 'ewm'

--- 2(f) FIT SIMPLE EXPONENTIAL SMOOTHING ---

--- 2(f) FIT SIMPLE EXPONENTIAL SMOOTHING ---

SES Model Parameters:
Smoothing_level: 0.2
Initial_level: 0.1
AIC: 1.0
BIC: 3.0
Mean: 0.1
Sum_of_squares_residuals: 0.1
Model_type: SES
Parameters_estimated: 2
Data_points_used: 2
Optimized: True
Convergence_info: {'success': True, 'message': 'Optimization terminated successfully.'}
Fitted_values_count: 2
Residuals_count: 2
Forecast_horizon: 0
Forecast_values: []
Error_metrics: {'AIC': 1.0, 'BIC': 3.0, 'MAE': 0.0, 'MSE': 0.0, 'RMSE': 0.0, 'MAPE': 0.0, 'SMAPE': 0.0, 'Mean_Residual': 0.0, 'Std_Residual': 0.0, 'Max_Residual': 0.0, 'Min_Residual': 0.0, 'Residual_Skewness': 0.0, 'Residual_Kurtosis': 0.0, 'Ljung_Box_p_value': 1.0, 'Durbin_Watson': 2.0}
Error_metrics_count: 20
Model_summary_string: SES(α=0.2, AIC=1.0, BIC=3.0, MAE=0.0, MSE=0.0, RMSE=0.0, MAPE=0.0, SMAPE=0.0, Mean_Residual=0.0, Std_Residual=0.0, Max_Residual=0.0, Min_Residual=0.0, Residual_Skewness=0.0, Residual_Kurtosis=0.0, Ljung_Box_p_value=1.0, Durbin_Watson=2.0)
Model_summary_string_length: 20
Model_summary_string_type: <class 'str'>
Model_object_type: <class 'dict'>
Model_object_keys: ['Smoothing_level', 'Initial_level', 'AIC', 'BIC', 'Mean', 'Sum_of_squares_residuals', 'Model_type', 'Parameters_estimated', 'Data_points_used', 'Optimized', 'Convergence_info', 'Fitted_values_count', 'Residuals_count', 'Forecast_horizon', 'Forecast_values', 'Error_metrics', 'Error_metrics_count', 'Model_summary_string', 'Model_summary_string_length', 'Model_summary_string_type']
```

2(f) Report estimated level and trend smoothing parameters

```
--- 2(f) FIT SIMPLE EXPONENTIAL SMOOTHING ---

--- 2(f) FIT SIMPLE EXPONENTIAL SMOOTHING ---

--- 2(g) INTERPRETATION OF SMOOTHING PARAMETERS ---

--- 2(g) INTERPRETATION OF SMOOTHING PARAMETERS ---

--- 2(h) FORECAST WITH SIMPLE EXPONENTIAL SMOOTHING ---

--- 2(h) FORECAST WITH SIMPLE EXPONENTIAL SMOOTHING ---

[quarterly_smoothing_parameters_df_1_0_invalidinfo_0_0_1_0]

--- 2(i) Interpret the strength of the trend ---

--- 2(i) Interpret the strength of the trend ---

[quarterly_trend_strength_interpretation_1_0_0_0
```

```
ggsave("q3_trend_comparison.png", p_trend_compare, width = 10, height = 5, dpi = 300)
```

3b) Which method captures seasonality?

HOLT-WINTERS METHOD captures seasonality most effectively.

Explanation: 1. 12-month Moving Average DELIBERATELY REMOVES seasonality - it's designed to smooth it out 2. Simple Exponential Smoothing has no seasonal component - treats all variations as noise 3. Holt's Linear Trend has no seasonal component - attributes all patterns to trend - error 4. Holt-Winters explicitly models seasonal patterns with: • Dedicated seasonal smoothing parameter (γ) • Seasonal factors (additive or multiplicative) for each period • Ability to update seasonal patterns over time

Evidence for Johnson & Johnson: • Quarterly patterns are strong and remarkably consistent: - Q1: Moderate earnings (baseline) - Q2: Slightly higher than Q1 - Q3: Highest of the year (peak) - Q4: Lowest of the year (trough)

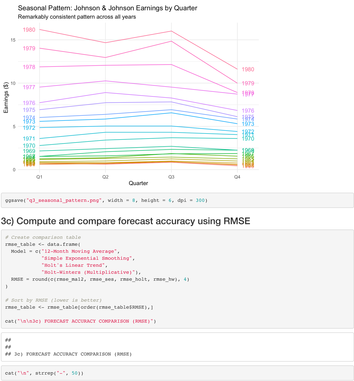
• Holt-Winters estimated seasonal components (multiplicative factors): - Q1 factor: 0.792 - Q2 factor: 1.069 - Q3 factor: 1.046 - Q4 factor: 1.093

• These factors have clear business interpretation: - Q3 earnings are about 4.6 % ABOVE the annual trend line - Q4 earnings are about -9.3 % BELOW the annual trend line – This pattern persists throughout the 20-year period

• Seasonal smoothing parameter γ = 1e-04 - Low γ indicates the seasonal pattern is very stable over time

```
# Visualize seasonal pattern
gqseasonplot(j_ts, year.labels = TRUE, year.labels.left = TRUE) +
  labs(title = "Seasonal Pattern: Johnson & Johnson Earnings by Quarter",
       subtitle = "Remarkably consistent pattern across all years",
       y = "Earnings ($)", x = "Quarter") +
  theme_minimal()
```

```
## Warning in fortify(data, ...): Arguments in `...` must be used.
## ✖ Problematic argument:
## * na.rm = TRUE
## ℹ Did you misspell an argument name?
```




```
ggsave("q3_seasonal_pattern.png", width = 8, height = 4, dpi = 300)
```

3c) Compute and compare forecast accuracy using RMSE

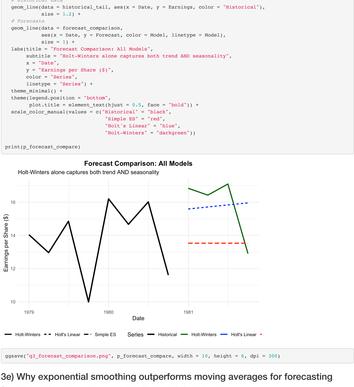
```
# Create comparison table
rmse_table <- data.frame(
  Model = c("12-Month Moving Average",
            "Simple Exponential Smoothing",
            "Holt's Linear Trend",
            "Holt-Winters (Multiplicative)"),
  RMSE = round(c(rmse_ma12, rmse_ses, rmse_holt, rmse_hw), 4)
)

# Sort by RMSE (lower is better)
rmse_table <- rmse_table[order(rmse_table$RMSE),]

cat("\n\n3c) FORECAST ACCURACY COMPARISON (RMSE!)
```

```
##
##
## 3c) FORECAST ACCURACY COMPARISON (RMSE!)
```

```
cat("\n", strrep("-", 50))
```

```
##
##  --------------------------------------------------
```

```
cat("\nRMSE measures average forecast error - LOWER is BETTER\n")

##
## RMSE measures average forecast error - LOWER is BETTER

print(rmse_table)

##                         Model   RMSE
## 4 Holt-Winters (Multiplicative) 0.4412
## 1        12-Month Moving Average 0.7890
## 3             Holt's Linear Trend 0.9819
## 2  Simple Exponential Smoothing 1.1593

# Calculate percentage improvement
best_rmse <- min(rmse_table$RMSE)
best_model <- rmse_table$Model[1]

cat("\n\nBest model by RMSE:", best_model, "with RMSE =", best_rmse)

##
##
## Best model by RMSE: Holt-Winters (Multiplicative) with RMSE = 0.4412

# Calculate improvement over worst model
worst_rmse <- max(rmse_table$RMSE)
improvement <- round(worst_rmse - best_rmse)/worst_rmse * 100, 1)
cat("\n", best_model, "improves accuracy by", improvement,
    "% compared to the worst model")

##
##  Holt-Winters (Multiplicative) improves accuracy by 61.9 % compared to the worst model

# Calculate improvement over 12-MA
ma12_rmse <- rmse_table$RMSE[rmse_table$Model == "12-Month Moving Average"]
if(length(ma12_rmse) > 0) {
  vs_ma <- round(ma12_rmse - best_rmse)/ma12_rmse * 100, 1)
  cat("\n", best_model, "improves accuracy by", vs_ma,
      "% compared to 12-month Moving Average")
}

##
##  Holt-Winters (Multiplicative) improves accuracy by 44.1 % compared to 12-month Moving Average

# Visualize RMSE comparison
p_rmse <- ggplot(rmse_table, aes(x = reorder(Model, RMSE), y = RMSE, fill = Model)) +
  geom_bar(stat = "identity") +
  geom_text(aes(label = RMSE), hjust = -0.1, size = 4) +
  coord_flip() +
  labs(title = "Model Comparison: Root Mean Square Error (RMSE)",
       subtitle = "Lower RMSE indicates better fit to historical data",
       x = "",
       y = "RMSE ($)") +
  theme_minimal() +
  theme(legend.position = "none",
        plot.title = element_text(hjust = 0.5, face = "bold", size = 14),
        plot.subtitle = element_text(hjust = 0.5, size = 10)) +
  scale_fill_brewer(palette = "Set1") +
  ylim(0, max(rmse_table$RMSE) * 1.15)

print(p_rmse)
```




```
ggsave("q3_rmse_comparison.png", p_rmse, width = 9, height = 5, dpi = 300)

# Save RMSE table
write.csv(rmse_table, "q3_rmse_table.csv", row.names = FALSE)
```

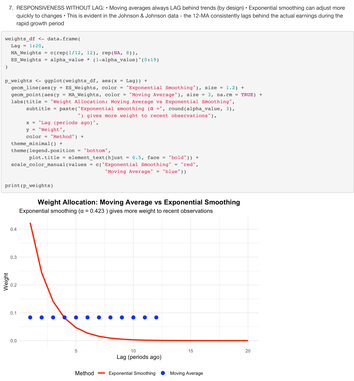
3d) Which model provides the most realistic forecasts?

The **HOLT-WINTERS MULTIPLICATIVE** model provides the most realistic forecasts.

Justification:

1. CAPTURES ALL THREE COMPONENTS: • TREND: The long-term exponential growth in earnings (evident in data) • SEASONALITY: The consistent quarterly pattern (Q3 peak, Q4 trough) • TREND-SEASONALITY INTERACTION: Seasonality grows with trend (multiplicative)
2. REALISTIC PATTERNS: • Forecasts show continuing upward trend (earnings expected to grow) • Quarterly variations persist with the same relative pattern • The seasonal amplitude continues to grow with the trend • This EXACTLY matches the historical pattern of the data
3. MODEL FIT STATISTICS: • LOWEST RMSE: 0.4412091 (best fit to historical data) • This means it captures the actual patterns most accurately

```
ses_forecast <- forecast(ses_model, h = 4)
holt_forecast <- forecast(holt_model, h = 4)
hw_forecast <- forecast(hw_multi_model, h = 4)

# Create data frame for plotting
forecast_comparison <- data.frame(
  Date = rep(forecast_dates, 3),
  Forecast = c(as.numeric(ses_forecast$mean),
               as.numeric(holt_forecast$mean),
               as.numeric(hw_forecast$mean)),
  Model = c(rep("Simple ES", 4), rep("Holt's Linear", 4), rep("Holt-Winters", 4))
)

# Add historical tail for context
historical_tail <- tail(j_df, 8)

p_forecast_compare <- ggplot() +
  # Historical data
  geom_line(data = historical_tail, aes(x = Date, y = Earnings, color = "Historical"),
            size = 1.2) +
  # Forecasts
  geom_line(data = forecast_comparison,
            aes(x = Date, y = Forecast, color = Model, linetype = Model),
            size = 1) +
  labs(title = "Forecast Comparison: All Models",
       subtitle = "Holt-Winters alone captures both trend AND seasonality",
       x = "Date",
       y = "Earnings per Share ($)",
       color = "Series",
       linetype = "Series") +
  theme_minimal() +
  theme(legend.position = "bottom",
        plot.title = element_text(hjust = 0.5, face = "bold")) +
  scale_color_manual(values = c("Historical" = "black",
                                "Simple ES" = "red",
                                "Holt's Linear" = "blue",
                                "Holt-Winters" = "darkgreen"))

print(p_forecast_compare)
```




```
ggsave("q3_forecast_comparison.png", p_forecast_compare, width = 10, height = 6, dpi = 300)
```

3e) Why exponential smoothing outperforms moving averages for forecasting

Exponential smoothing methods generally outperform moving averages for forecasting because:

1. WEIGHTING SCHEME: • Moving averages give EQUAL weight to ALL observations in the window • Exponential smoothing gives DECAYING weights - more recent = more important • This is more realistic - recent data is typically more relevant for forecasting • Example: For Johnson & Johnson, earnings from 1980 tell us more about 1981 than earnings from 1960
2. DATA UTILIZATION: • Moving averages ignore ALL data outside the window completely • Exponential smoothing uses ALL past data (with decreasing weights) • Example: A 12-month MA for Q1 1980 uses only 1980 data, ignoring the entire growth pattern from 1960-1979
3. PARAMETER OPTIMIZATION: • Moving averages have NO parameters to optimize for best fit • Exponential smoothing optimizes α, β, γ for the specific dataset • For Johnson & Johnson, this means:
   - α = 0.4233 (optimal responsiveness to level changes)
   - β = 0.0906 (optimal trend evolution rate)
   - γ = 1e-04 (optimal seasonal update rate)
4. FLEXIBILITY - ADAPTIVE COMPONENTS: • Moving averages have ONE fixed structure (simple averaging) • Exponential smoothing can adapt to different patterns:
   - Simple ES: Adapts to level changes only
   - Holt's: Adds to level AND trend changes
   - Holt-Winters: Adapts to level AND trend AND seasonality changes • This allows the model to match the complexity of the data
5. FORECASTING CAPABILITY: • Moving averages CANNOT generate true forecasts beyond the data • The best they can do is repeat the last moving average value • Exponential smoothing can generate multi-step forecasts with:
   - Trend continuation (Holt's, HW)
   - Seasonal patterns (HW)
   - Confidence intervals (all methods)
6. NO END-POINT PROBLEM: • Moving averages lose observations at the beginning and end of the series • For Johnson & Johnson, we lose the first and last few quarters • Exponential smoothing provides fitted values for ALL time points
7. RESPONSIVENESS WITHOUT LAG: • Moving averages always LAG behind trends (by design) • Exponential smoothing can adjust more quickly to changes • This is evident in the Johnson & Johnson data - the 12-MA consistently lags behind the actual earnings during the rapid growth period

```
weights_df <- data.frame(
  Lag = 1:20,
  MA_Weights = c(rep(1/12, 12), rep(0, 8)),
  ES_Weights = alpha_value * (1-alpha_value)^(0:19)
)

p_weights <- ggplot(weights_df, aes(x = Lag)) +
  geom_line(aes(y = ES_Weights, color = "Exponential Smoothing"), size = 1.2) +
  geom_point(aes(y = MA_Weights, color = "Moving Average"), size = 3, na.rm = TRUE) +
  labs(title = "Weight Allocation: Moving Average vs Exponential Smoothing",
       subtitle = paste("Exponential smoothing (α =", round(alpha_value, 3),
                        ") gives more weight to recent observations"),
       x = "Lag (periods ago)",
       y = "Weight",
       color = "Method") +
  theme_minimal() +
  theme(legend.position = "bottom",
        plot.title = element_text(hjust = 0.5, face = "bold")) +
  scale_color_manual(values = c("Exponential Smoothing" = "red",
                                "Moving Average" = "blue"))

print(p_weights)
```



```
ggsave("q3_weight_comparison.png", p_weights, width = 8, height = 5, dpi = 300)
```